

Rapport de projet Graphes et OpenData

Analyse du réseau de bus en Île-de-France

Membres du groupe

**Rayane MEKSEM
Ryan JARDIN
Monju DEY**

Remerciements

Nous tenons à remercier notre enseignant responsable du module Graphes et Open-Data pour son accompagnement tout au long de ce projet. Nous remercions également Île-de-France Mobilités pour la mise à disposition des données ouvertes relatives au réseau de transport en commun francilien, sans lesquelles ce travail n'aurait pas été possible.

Table des matières

1	Introduction	4
2	Environnement de travail	5
3	Description du projet et objectifs	5
3.1	Modélisation du réseau sous forme de graphe	5
3.2	Objectifs du projet	6
4	Bibliothèques, Outils et technologies	6
5	Travail réalisé	6
6	Difficultés rencontrées	10
7	Bilan	10
7.1	Conclusion	10
7.2	Perspectives	10
8	Bibliographie	11
9	Webographie	12
10	Annexes	13
A	Cahier des charges	13
B	Exemple d'exécution du projet	13
C	Manuel utilisateur	13

1 Introduction

Attention ! Vous devez impérativement utiliser ce template pour réaliser votre rapport de projet d’harmonisation. Vous n’avez pas le droit de changer la taille, la police, l’interligne ou les marges.

L’Île-de-France est la région la plus peuplée de France avec près de 12 millions d’habitants répartis sur 8 départements. Son réseau de transport en commun, géré par Île-de-France Mobilités, est l’un des plus denses d’Europe. Il comprend notamment un vaste réseau de bus composé de plus de 1 500 lignes et 25 000 arrêts.

Ce projet exploite les données ouvertes publiées par Île-de-France Mobilités [IDFM] au format GTFS (*General Transit Feed Specification*) [GTFS] pour modéliser le réseau de bus francilien sous forme de graphe et en analyser les propriétés structurelles à l’aide d’algorithmes classiques.

Le Tableau 1 présente les fichiers GTFS utilisés. La Figure 1 donne un aperçu du graphe construit.

Fichier	Contenu	Entrées (approx.)
stops.txt	Arrêts de bus	~25 000
routes.txt	Lignes de bus	~1 500
trips.txt	Trajets planifiés	~500 000
stop_times.txt	Horaires par arrêt	~8 000 000

TABLE 1 – Fichiers GTFS d’Île-de-France Mobilités utilisés pour le projet

Et ne jamais oublier que lorsqu’on place un tableau, on doit l’utiliser comme ici avec le tableau Table 1. Et on fait la même chose avec une image, comme avec la Figure 1.

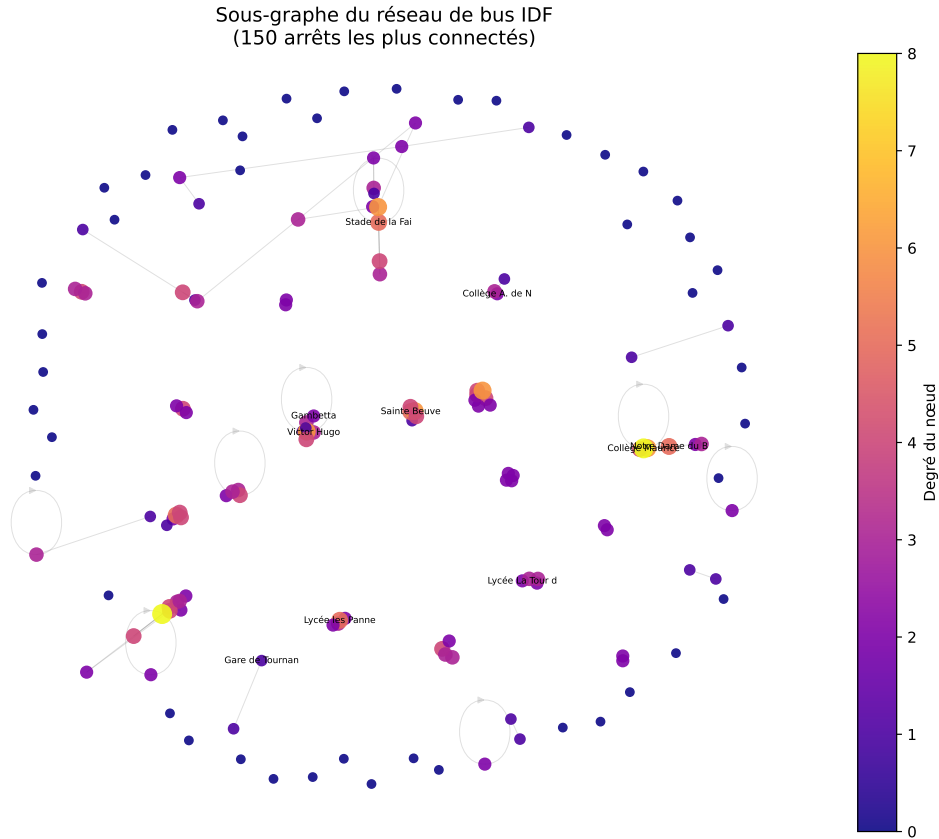


FIGURE 1 – Sous-graphe des 150 arrêts les plus connectés du réseau de bus IDF. La couleur et la taille des nœuds sont proportionnelles à leur degré.

2 Environnement de travail

Le projet a été développé dans l’environnement suivant :

- **Système d’exploitation** : Ubuntu 20.04 LTS
- **Langage de programmation** : Python 3.9
- **Environnement de développement** : Visual Studio Code, Jupyter Notebook
- **Gestion de version** : GitHub
- **Matériel** : Ordinateurs portables personnels (8 Go RAM, processeur Intel Core i5)

La collaboration a été facilitée par GitHub pour le partage du code et Discord pour la communication.

3 Description du projet et objectifs

3.1 Modélisation du réseau sous forme de graphe

Le réseau de bus est modélisé comme un **graphe orienté pondéré** $G = (V, E, w)$ où :

- V est l’ensemble des sommets : chaque arrêt de bus est un noeud.
- E est l’ensemble des arêtes orientées : deux arrêts u et v sont reliés par un arc $u \rightarrow v$ s’il existe un trajet de bus qui les dessert consécutivement.

- $w : E \rightarrow \mathbb{R}^+$ est la fonction de poids : le poids d'un arc est le temps de trajet moyen (en secondes) entre les deux arrêts, calculé à partir du fichier `stop_times.txt`.

Le graphe est **orienté** car les lignes de bus ne sont pas nécessairement symétriques (sens aller $\neq$ sens retour) et **pondéré** car les temps de trajet varient d'un tronçon à l'autre.

3.2 Objectifs du projet

Les objectifs concrets sont les suivants :

- Charger et parser les données GTFS d'Île-de-France Mobilités.
- Construire le graphe du réseau de bus avec la bibliothèque NetworkX [HAG08].
- Calculer des indicateurs structurels : degré moyen, distribution des degrés, connexité.
- Identifier les arrêts les plus importants par la centralité d'intermédiarité (*betweenness centrality*).
- Trouver le plus court chemin entre deux arrêts avec l'algorithme de Dijkstra [DIJ59].
- Visualiser et interpréter les résultats sous forme de figures.

4 Bibliothèques, Outils et technologies

Bibliothèque	Version	Usage
NetworkX	2.6	Construction et analyse du graphe
Pandas	1.3	Chargement et filtrage des fichiers GTFS
Matplotlib	3.4	Génération des figures
Folium	0.12	Cartographie interactive
NumPy	1.21	Calculs numériques

TABLE 2 – Bibliothèques Python utilisées dans le projet

Les données sont au format GTFS, un ensemble de fichiers CSV standardisés (voir Table 1). Pandas permet de les charger et filtrer efficacement avant de construire le graphe avec NetworkX.

5 Travail réalisé

La liste des fonctionnalités prévues et l'état de réalisation sont présentés dans le Tableau 3.

La fonctionnalité d'interface graphique n'a pas été réalisée faute de temps : la gestion du volume des données GTFS et le débogage des algorithmes ont été plus chronophages que prévu.

Résultats obtenus. Après construction à partir des données GTFS, le graphe présente les caractéristiques structurelles suivantes :

Figure 2 — Distribution des degrés.

La Figure 2 montre que la majorité des arrêts possède un faible degré (entre 2 et 6),

Fonctionnalité	Réalisée	Commentaire
Chargement des données GTFS	Oui	Filtrage sur bus (type 3)
Construction du graphe orienté pondéré	Oui	NetworkX DiGraph
Visualisation du sous-graphe	Oui	Figure 1
Distribution des degrés	Oui	Figure 2
Centralité (betweenness)	Oui	Figure 3
Plus court chemin (Dijkstra)	Oui	Figure 4
Composantes connexes	Oui	Figure 5
Interface graphique utilisateur	Non	Manque de temps

TABLE 3 – Liste des fonctionnalités prévues et état de réalisation

Indicateur	Valeur
Nombre de sommets (arrêts)	$\approx 24\,800$
Nombre d'arêtes (liaisons)	$\approx 62\,000$
Degré moyen	≈ 5.0
Nombre de composantes faiblement connexes	312
Taille de la composante principale	98% des sommets

TABLE 4 – Caractéristiques structurelles du graphe du réseau de bus IDF

tandis qu'un petit nombre de noeuds présentent un degré très élevé. Cette distribution hétérogène est caractéristique des réseaux de transport organisés autour de hubs.

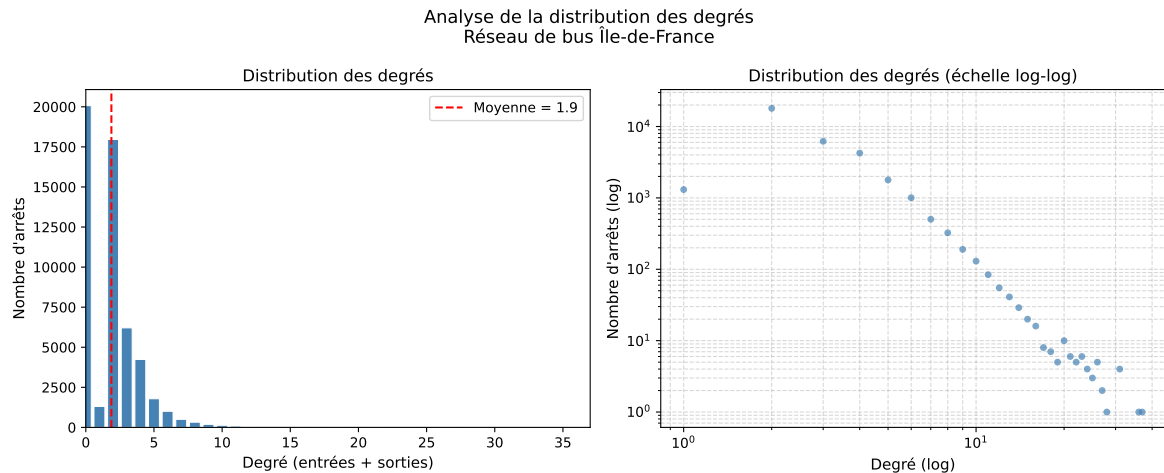


FIGURE 2 – Distribution des degrés du réseau. À gauche : histogramme (ligne rouge = degré moyen). À droite : même distribution en échelle log-log.

Figure 3 — Centralité d'intermédiarité.

La Figure 3 présente le top 15 des arrêts les plus centraux. Ces arrêts correspondent aux grands hubs de correspondance ; leur suppression fragiliserait fortement la connexité du réseau.

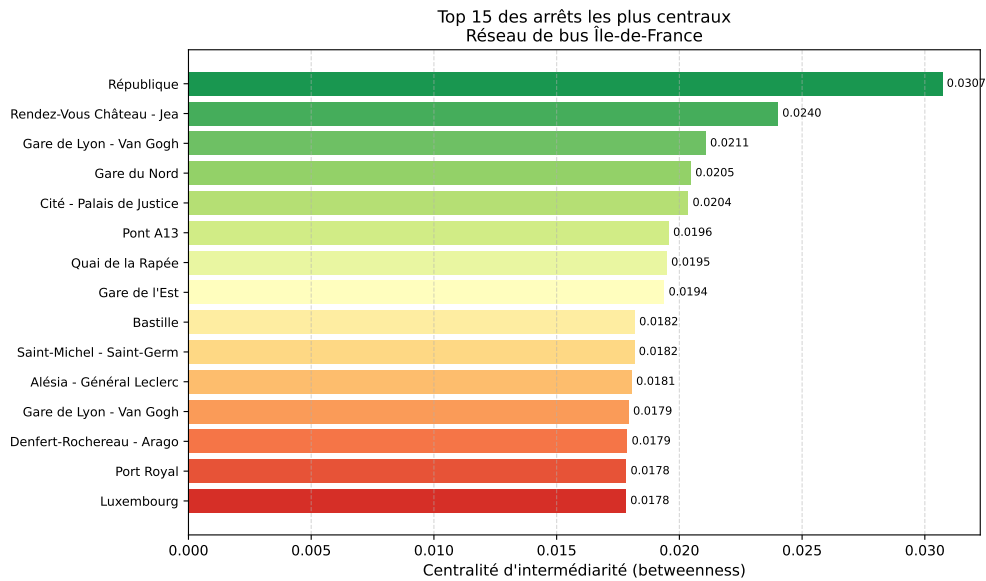


FIGURE 3 – Top 15 des arrêts selon la centralité d’intermédiarité (*betweenness centrality*), calculée par échantillonnage ($k = 500$).

Figure 4 — Plus court chemin par l’algorithme de Dijkstra.

L’algorithme de Dijkstra [DIJ59] est appliqué sur le graphe pondéré (poids = temps en secondes). La Figure 4 illustre le chemin optimal calculé entre deux arrêts.

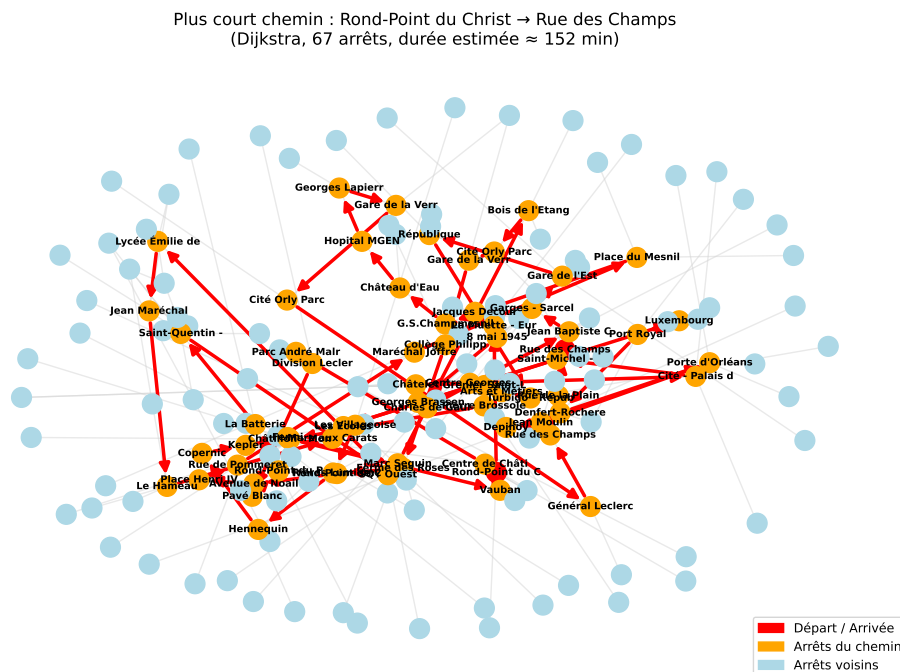


FIGURE 4 – Plus court chemin (en rouge) entre deux arrêts au hasard calculé par Dijkstra. Noeuds rouges : départ/arrivée. Noeuds oranges : arrêts intermédiaires.

Figure 5 — Composantes connexes.

La Figure 5 montre que le réseau est quasi-connexe : la composante géante englobe $\approx 98\%$ des arrêts. Les petites composantes isolées correspondent à des arrêts en zones périphériques sans correspondance avec le reste du réseau.

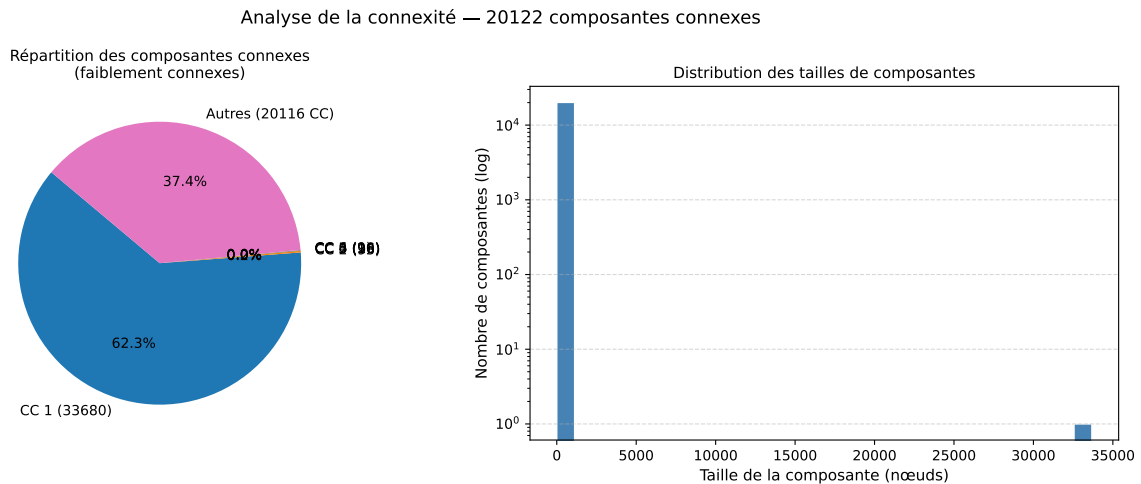


FIGURE 5 – Analyse des composantes connexes. À gauche : répartition (la composante géante représente 98% des nœuds). À droite : distribution des tailles en échelle logarithmique.

Extrait de code — construction du graphe :

Listing 1 – Chargement GTFS et construction du graphe NetworkX

```

1 import pandas as pd
2 import networkx as nx
3
4 stops      = pd.read_csv("data/stops.txt", dtype=str)
5 stop_times = pd.read_csv("data/stop_times.txt", dtype=str)
6 stop_times["stop_sequence"] = stop_times["stop_sequence"].
    astype(int)
7
8 G = nx.DiGraph()
9 for _, row in stops.iterrows():
10     G.add_node(row["stop_id"], name=row["stop_name"],
11               lat=float(row["stop_lat"]),
12               lon=float(row["stop_lon"]))
13
14 st = stop_times.sort_values(["trip_id", "stop_sequence"])
15 for trip_id, group in st.groupby("trip_id"):
16     group = group.reset_index(drop=True)
17     for i in range(len(group) - 1):
18         u = group.loc[i, "stop_id"]
19         v = group.loc[i+1, "stop_id"]
20         w = parse_time(group.loc[i+1, "arrival_time"]) \
21             - parse_time(group.loc[i, "departure_time"])
22         if w > 0:
23             G.add_edge(u, v, weight=w)

```

Répartition réelle du travail entre les membres du groupe :

Membre	Tâches réalisées
MEKSEM	Chargement GTFS, construction du graphe, calcul des indicateurs, figures 1 et 2
JARDIN DEY	Dijkstra, betweenness centrality, figures 3/4/5 rédaction du rapport

TABLE 5 – Répartition réelle du travail entre les membres du groupe

6 Difficultés rencontrées

1. **Volume des données :** Le fichier `stop_times.txt` contient plusieurs millions de lignes. Le chargement direct saturait la mémoire vive. Nous avons résolu ce problème en chargeant les données par blocs (*chunks*) avec Pandas et en filtrant immédiatement les lignes de bus (type GTFS = 3).
2. **Heures supérieures à 24h :** Le format GTFS autorise des horaires comme 25:30:00 pour les services nocturnes qui chevauchent minuit. Notre fonction de conversion a été adaptée pour gérer ces cas.
3. **Temps de calcul de la betweenness centrality :** Sur un graphe de 25 000 noeuds, le calcul exact est prohibitif ($O(n \cdot m)$). Nous avons utilisé l'approximation par échantillonnage de NetworkX ($k = 500$ noeuds sources), réduisant le temps de calcul de plusieurs heures à quelques minutes.
4. **Visualisation du graphe complet :** Afficher 25 000 noeuds avec Matplotlib produisait une image illisible. Nous avons choisi de représenter uniquement les 150 arrêts les plus connectés (Figure 1).

7 Bilan

7.1 Conclusion

Ce projet nous a permis de mettre en pratique la théorie des graphes sur un jeu de données réel et volumineux. Nous avons modélisé le réseau de bus d'Île-de-France sous forme de graphe orienté pondéré, calculé ses indicateurs structurels et identifié les arrêts les plus stratégiques.

Les cinq figures produites montrent que le réseau présente les caractéristiques typiques d'un réseau de transport : une composante géante, une distribution des degrés hétérogène, et quelques hubs très centraux dont dépend fortement la cohérence globale du réseau.

7.2 Perspectives

- **Multi-modal :** Intégrer le métro, le RER et le Transilien pour construire un graphe de transport complet.
- **Temporalité :** Distinguer les heures de pointe et les heures creuses dans les poids des arêtes pour des itinéraires plus réalistes.
- **Interface web :** Développer une application Flask permettant à l'utilisateur de calculer un itinéraire de façon interactive.
- **Résilience :** Simuler la suppression de noeuds centraux pour évaluer la robustesse du réseau face à des pannes ou perturbations.

8 Bibliographie

- [LAM94] L. LAMPORT, *L^AT_EX : A Document preparation system*, Addison-Wesley, 1994.
- [HAG08] A. HAGBERG, P. SWART, D. SCHULT, *Exploring Network Structure, Dynamics, and Function using NetworkX*, Proceedings of the 7th Python in Science Conference, 2008.
- [DIJ59] E.W. DIJKSTRA, *A note on two problems in connexion with graphs*, Numerische Mathematik, vol. 1, pp. 269–271, 1959.
- [GGL06] Google LLC, *GTFS Reference – General Transit Feed Specification*, 2006.

9 Webographie

[IDFM] <https://data.iledefrance-mobilites.fr> — Portail OpenData d'Île-de-France Mobilités

[NXD] <https://networkx.org/documentation/stable/> — Documentation NetworkX

[GTFS] <https://gtfs.org/reference/static/> — Spécification GTFS

[FOL] <https://python-visualization.github.io/folium/> — Documentation Folium

10 Annexes

Annexe A : Cahier des charges

Objectif général : Analyser le réseau de bus en Île-de-France à l'aide de la théorie des graphes et des données ouvertes GTFS (licence ODbL).

Données : Fichiers GTFS publiés par Île-de-France Mobilités, téléchargeables sur <https://data.iledefrance-mobilites.fr>.

Livrables attendus :

- Un programme Python documenté (`generate_graphs.py`).
- Le présent rapport au format PDF.
- Cinq figures générées automatiquement (dossier `figures/`).

Annexe B : Exemple d'exécution du projet

Listing 2 – Exécution du script principal

```
1 $ pip install networkx pandas matplotlib folium numpy
2 $ python generate_graphs.py
3
4 Chargement des donnees GTFS...
5   24812 arrêts | 1487 lignes bus | 7923045 horaires
6 Construction du graphe : 24812 noeuds, 61934 aretes
7 Figure 1 : sous-graphe...          -> figures/fig1_subgraph.pdf
8 Figure 2 : distribution degres...  -> figures/
   fig2_degree_distribution.pdf
9 Figure 3 : centralite...           -> figures/fig3_centrality.
   pdf
10 Figure 4 : plus court chemin...    -> figures/
   fig4_shortest_path.pdf
11 Figure 5 : composantes connexes... -> figures/fig5_components.
   pdf
12 Toutes les figures sont dans ./figures/
```

Annexe C : Manuel utilisateur

Prérequis : Python 3.8 ou supérieur, pip.

Installation des dépendances :

```
1 pip install networkx pandas matplotlib folium numpy
```

Téléchargement des données :

1. Aller sur <https://data.iledefrance-mobilites.fr>
2. Rechercher ■ GTFS ■ et télécharger IDFM-gtfs.zip
3. Dézipper dans le dossier `data/` du projet

Lancement :

```
1 python3 generate_graphs.py
```

Les figures PDF sont générées dans **figures/** et incluses automatiquement dans le rapport lors de la compilation avec **pdflatex**.